Laxmi Charitable Trust's

Sheth L.U.J College of Arts & Sir M.V. College of Science and Commerce

Department of Information Technology (B.Sc.I.T Semester IV)

Dot .Net Core and Progressive Web Application Practical

Practical – III

Roll No.: S044	Name: SACHIN SINGH
Class: TYIT	Batch:1
Date of Assignment: 14/07/2026	Date/Time of Submission: 14/07/2026

Aim(A) : Create methods add(). multiply(), subtract() ,divide() with suitable parameters and call these methods using concept of C# delegate.

Algorithm:

Step 1 Start
Step 2: Declare the delegate : delegate void Arithematic_cal(int a,int b);
Step 3: Create methods having the same signature
Step 4: Create a delegate object : Arithematic_cal ac;
Step 5: Assign a method to the delegate : ac =c.add;
Step 6: Invoke the delegate : ac(10,20);
Step 7 : Stop

Code:

```
using System;
delegate void Arithematic_cal(int a,int b);
class Calculator{
    public void add(int a, int b){
        Console.WriteLine("Additon : "+(a+b));
    }
     public void sub(int a, int b){
        Console.WriteLine("Subtraction : "+(a-b));
    }
     public void mul(int a, int b){
        Console.WriteLine("Multiplication : "+(a*b));
    }
     public void div(int a, int b){
        Console.WriteLine("Diviosn : "+(a/b));
    }
}
public class HelloWorld
{
    static void Main(string[] args)
    {
        Calculator c = new Calculator();
        Arithematic_cal ac;
        ac =c.add;
        ac(10,20);
```

```
        ac =c.sub;
        ac(10,20);
        ac =c.mul;
        ac(10,20);
        ac =c.div;
        ac(10,20);
    }
}
```

Output:

```
Additon : 30
Subtraction : -10
Multiplication : 200
Diviosn : 0
```

Aim(B) : Write a program using multicast delegate.

Algorithm:

Step 1 Start

Step 2: Declare the delegate : delegate void Message();

Step 3: Create methods having the same signature : public void Hello()

Step 4: Create a delegate object and assign method : Message msg = mp.Hello

Step 5: Multicast the delegate : msg +=mp.Msg;

Step 6 : Stop

Code:

```
using System;
delegate void Message();
class Msg_Print{
    public void Hello(){
        Console.WriteLine("HELLO TO C# CODING ");
    }
     public void Msg(){
        Console.WriteLine("This is your welcome Message ");
    }
     public void Thank(){
        Console.WriteLine("Thank you for Coding ");
     }
}

public class Program
{
    static void Main(string[] args)
    {
        Msg_Print mp = new Msg_Print();
        Message msg = mp.Hello;
        msg +=mp.Msg;
        msg +=mp.Thank;
```

```
        msg();
    }
}
```

Output:

```
HELLO TO C# CODING :
This is your welcome Message
Thank you for Coding
```

Aim(C) : Create a class BankAccount with AccountNumber and Balance. Implement property for Balance with validation (Balance cannot be negative).

Algorithm:

Step 1 Start
Step 2: Declare the class :
Step 3: Create methods having the same signature and use get,set functions
Step 4: Create a delegate object :
Step 5: Assign a method to the delegate :
Step 6: Invoke the delegate :
Step 7 : Stop

Code:

```
using System;
class Bank{
    public int Acc_No{get;set;}
        private double bal;
        public double Bal{
            get{
                return bal;
            }
            set{
                if(value >=0){
                    bal=value;
                }
                else{
                    Console.WriteLine("Balance cannot be Negative");
                }
            }
        }
}
class Program
{
    static void Main(string[] args)
    {
        Bank b = new Bank();
        b.Acc_No = 101;
        b.Bal = 5000;
```

```
        Console.WriteLine("Account Number : "+b.Acc_No);
        Console.WriteLine("Balance : "+b.Bal);
        b.Bal=-1000;

    }
}
```

Output:

```
Account Number : 101
Balance : 5000
Balance cannot be Negative
```